Contrastive Forecasting: Initial Technical Report

Jeremy Sylvain Cochoy
jeremy.cochoy@gmail.com

Abstract

Current time-series foundation models forecast in value space, following the language-modelling recipe of next-token prediction. Because the observed series is one realisation of a stochastic process, a value-space loss asks the model to reproduce intrinsic noise that no model can predict. We propose *Contrastive Forecasting*, a training framework in which an encoder and a causal forecaster are trained jointly to predict the next patch in latent space through contrastive learning, with negatives drawn along the cross-temporal, cross-channel and cross-batch axes. The framework is decoupled from the choice of encoder and forecaster architecture. On synthetic stationary ARMA processes, the latents produced by the backbone carry enough information about the generating process for a small head to recover its parameters.

1 Introduction

Time-series forecasting has converged on a foundation-model recipe inherited from language modelling: patched inputs, a transformer backbone, and a head that forecasts the next patch in value space. TimesFM [Das et al., 2024] is the canonical example, and recent successors [Ansari et al., 2024, Woo et al., 2024, Shi et al., 2024, Liu et al., 2025, 2026] extend the recipe along various axes while keeping the value-space target. As Ilya Sutskever has argued for large language models, next-token prediction is far less narrow than it looks: it forces the model to encode grammar, style, world knowledge and arithmetic, since each is a sub-task of the same prediction problem. The next-patch analogue should inherit this property for time series. The choice of value space, however, is structurally suboptimal: the observed series is one realisation of a stochastic process, so a value-space loss asks the model to reproduce the intrinsic noise of that process. No model can succeed at that task, and the training loss never reaches zero, even on processes whose predictable component the model captures perfectly.

A second line of work shifts the prediction target from raw values to a learned representation. The Joint Embedding Predictive Architecture (JEPA) [Assran et al., 2023], originally introduced for vision, encodes a context block with one network and a target block with another, and trains a predictor to map the context latent to the target latent. The recent time-series adaptations [Verdenius et al., 2024, Ennadir et al., 2025] apply the same idea to univariate series. By predicting an abstract feature of the future rather than its raw values, the model is no longer punished for missing the noise. The siamese architecture used by JEPA, however, is at risk of collapse by construction: the constant function is a global minimum of its loss. Contrastive learning with explicit negatives is the classical anti-collapse remedy, and has been used at scale for time-series representation learning (CPC [van den Oord et al., 2018], TS2Vec [Yue et al., 2022]), but to our knowledge no published time-series foundation model uses it as a primary forecasting objective.

We ask whether a time-series forecasting model can be trained directly in latent space with an explicit contrastive objective, in such a way that the latent it learns encodes the underlying data-generating process. The rest of the paper proposes a candidate architecture and tests this question on a controlled synthetic distribution, where the generating parameters are known and recoverability can be checked directly.

Preprint. Under review.

The contributions of the paper are the following.

- A training framework, *Contrastive Forecasting*, in which an encoder and a causal forecaster are trained jointly with an InfoNCE objective on cross-temporal, cross-channel and cross-batch negatives (Section 3). The framework is decoupled from the choice of encoder and forecaster architecture.
- Empirical evidence that the resulting latents encode the parameters of the data-generating process. On synthetic ARMA, a frozen-backbone recovery head retrieves the AR and MA coefficients with an improvement ratio well above the all-zero baseline (Sections 4 and 5).
- An architecture-search trail for both the encoder and the forecaster, with the design decisions reported in Appendix D.

2 Related Work

Time-series forecasting and the foundation-model era. Per-series statistical models (ARIMA, exponential smoothing, state-space) dominated forecasting until global deep models such as DeepAR [Salinas et al., 2020] and N-BEATS [Oreshkin et al., 2020] showed that one network trained over many series can outperform per-series fits. Transformers were beaten by simple linear baselines on these benchmarks until PatchTST [Nie et al., 2023] introduced patch tokenization: contiguous values are grouped into a single token, the attention cost drops by the patch length, and the model gains a useful local prior. TiDE [Das et al., 2023] pursued an MLP-only line in parallel, with a residual block over patched inputs that is structurally close to the encoder we use here. TimesFM [Das et al., 2024] is the canonical decoder-only patched TS foundation model: pretrained on roughly 100B time points, its zero-shot accuracy is on par with supervised specialists on Monash, Darts and ETT. The follow-ups extend the recipe along different axes: Chronos [Ansari et al., 2024] tokenizes values into a discrete vocabulary and trains a T5 encoder-decoder over them; MOIRAI [Woo et al., 2024] masks patches and reconstructs them with an encoder; Time-MoE [Shi et al., 2024] scales to a billion parameters with sparse routing; Sundial [Liu et al., 2025] replaces categorical heads with continuous flow matching; Timer-S1 [Liu et al., 2026] currently leads GIFT-Eval. All of them forecast in value space, whether the head is regressive, categorical or generative.

Contrastive representation learning for time series. A separate line learns time-series representations by discrimination rather than reconstruction. Contrastive Predictive Coding [van den Oord et al., 2018] encodes a series with a strided CNN, summarises the past with a GRU, and trains the encoder to assign higher score to the true future latent than to a batch of negatives. The loss, InfoNCE [van den Oord et al., 2018], is a tractable lower bound on the mutual information between the context and the future, so the encoder is forced to keep whatever in the past predicts the future. The time-series successors keep the contrastive backbone but rework what counts as a positive pair: subseries consistency in T-Loss [Franceschi et al., 2019], temporal neighbourhoods in TNC [Tonekaboni et al., 2021], contextual consistency under masking and random crops with hierarchical contrast in TS2Vec [Yue et al., 2022]. These methods produce strong representations for classification and anomaly detection on UCR and UEA, but no current TS foundation model uses contrastive learning as the primary pretraining objective.

Latent-space prediction without contrast. A third line, originating in vision, predicts in latent space rather than in input space. The Joint Embedding Predictive Architecture [Assran et al., 2023] encodes a context block with one network, encodes the target block with a second network whose weights are an exponential moving average of the first, and trains a predictor to map the context latent to the target latent. The loss is L2 in latent space. Collapse is avoided by the combination of the asymmetric predictor and the stop-gradient on the EMA target rather than by contrast: there are no negatives. Two recent papers carry the idea to time series. LaT-PFN [Verdenius et al., 2024] combines JEPA with a prior-data fitted network for in-context forecasting, and TS-JEPA [Ennadir et al., 2025] adapts the patch-token JEPA recipe to univariate series, with competitive results on classification and long-term forecasting. Masked-reconstruction TS models such as MOMENT [Goswami et al., 2024] and MOIRAI [Woo et al., 2024] sit closer to JEPA than tokenized or autoregressive lines do, since they too predict masked content from visible context, but their target remains the raw values rather than their representation.

Positioning. Decoder-only transformers trained to predict the next token have become the dominant architecture in language. As Sutskever has argued, this objective is far less narrow than it looks: predicting the next token correctly requires the model to encode grammar, style, world knowledge and arithmetic, since each is a sub-task of the same prediction problem. We want to recover this property for time series, with a single decoder-only model that predicts the next patch and learns whatever structure is useful along the way, but without committing to value-space prediction. The observed series is a sample from a distribution generated by a stochastic process, so a value-space loss penalises the model for failing to predict intrinsic noise that no model can forecast. Predicting in latent space lets the target be a representation of the distribution of plausible futures, with the observed value treated as one of its samples, and the model is not forced to fit details that carry no semantic content.

Contrastive forecasting is our attempt at this combination. We forecast the next patch in latent space, as in JEPA, but enforce non-collapse with explicit InfoNCE [van den Oord et al., 2018] negatives, as in CPC and TS2Vec. JEPA is a siamese architecture and is therefore at risk of collapse by construction: the predictor can drive the L2 loss to zero by mapping every context to the same point. With an InfoNCE loss the constant function is instead one of the least stable solutions, since every sample is pushed away from every other sample in latent space, and the collapsed embedding is actively repelled. Compared to CPC and its successors, the target is forecasting rather than representation transfer for classification. To our knowledge, no published work has measured latent-space forecasting with explicit negatives at TS-FM scale.

3 Contrastive Forecasting

Contrastive Forecasting consists of a per-patch encoder and a causal forecaster, trained jointly with a contrastive loss in latent space. The subsections below define the patching, the encoder, the forecaster, and the loss in turn.

3.1 Architecture

We work with a multivariate time series of shape $T \times C \times F$, where T is the number of raw timesteps, C is the number of channels, and F is the number of features per timestep. In the experiments reported here $F = 1$, but the formalism does not change when $F > 1$. The series is cut into non-overlapping patches of width W along the time axis, so that the patched input has shape $T_p \times C \times W \times F$ with $T_p = T/W$. Each patch is encoded independently into a latent $h[t, c] \in \mathbb{R}^H$, and we project it onto the Euclidean unit sphere by L^2-normalising, so that $\|h[t, c]\| = 1$.

The choice of W is dataset-dependent. Values in $\{16, 32, 64, 128\}$ all appear in the recent foundation-model literature. We use $W = 32$ in the experiments reported here, following the patch size used by TimesFM [Das et al., 2024]. To minimise compute, we set the stride between consecutive patches to the patch width, giving non-overlapping windows that cover the full sequence. A stride of 1 is also of interest, since it gives one latent per timestep, provided the loss is adjusted to align each forecast with the correct future patch.

Our architecture has two models: an encoder and a forecaster. The encoder maps a patch to a latent; the forecaster maps the sequence of past latents to a forecast of the next latent. We write $f[t, c]$ for the forecaster's output at position t and channel c. The training objective is to make $f[t, c]$ as close as possible to $h[t + 1, c]$ under some dissimilarity D,

$$D\big(f[t, c],\ h[t + 1, c]\big) \to 0.$$

Placing the latents on the unit sphere is a deliberate choice: it lets us take D to be the angular distance, measured through its proxy the inner product (Section 3.4). The representation also benefits from living on a closed surface.

3.2 Encoder

The first encoder we tried is the residual block used by TiDE [Das et al., 2023] and TimesFM [Das et al., 2024]: a small MLP (linear, ReLU, dropout, linear) summed with a linear skip from the input, followed by a layer normalisation. The input dimension is W and the output dimension is H.

The encoder we currently use is a bidirectional GRU. Each patch is read as a sequence of W scalar timesteps, which gives the encoder a chance to model the local temporal structure that an MLP collapses into a single linear projection. The forward and backward final hidden states (128 units each in our setup) are concatenated to a 256-dim vector and projected to H, summed with a linear skip from the raw patch, and passed through a layer normalisation.

The encoder design space remains largely unexplored. In theory the encoder could be as complex as a transformer, if encoding a patch into a useful latent turns out to be a hard problem in its own right. Finding the proper encoder is a potential future-work axis.

3.3 Forecaster

The forecaster is a causal transformer decoder. It has L layers and hidden width H, with L and H controlling the depth and the capacity. Each layer first applies a small depthwise causal convolution (kernel size 3) directly to the stream, then a self-attention block, then a feed-forward network. The attention and feed-forward sub-layers follow the standard pre-norm pattern with residual connections; the depthwise convolution is applied without a residual wrap.

The convolution follows the observation of Elhage et al. [Elhage et al., 2021] that a short-range depthwise convolution lets induction-head-like circuits form without the second attention layer that the canonical induction circuit requires. This significantly improves the performance of small transformers, the regime we operate in.

3.4 Loss function

For two unit-norm latents we score their similarity by their inner product, which is the cosine of the angle between them and is convenient to compute.

We index each latent and each forecast by the batch element $b \in B$, the patch position $t \in T_p$, and the channel $c \in C$. For every anchor (b, t, c), the positive pair is the forecast $f[b, t, c]$ paired with the future latent $h[b, t+1, c]$.

The negative set is built by varying anchors along three independent axes: cross-temporal ($t \neq t'$), cross-channel ($c \neq c'$), and cross-batch ($b \neq b'$). Each axis can be combined with the others, and each pair can mix encoded latents h and forecasts f. The loss uses three specific pair families.

- **Cross-temporal** ($\mathcal{N}_{\mathrm{ct}}$): two encoded latents from the same series and channel at consecutive times, $\big(h[b, t, c],\ h[b, t+1, c]\big)$. Per anchor, 1 candidate.
- **Cross-channel** ($\mathcal{N}_{\mathrm{cc}}$): two encoded latents from the same series at the same time, with different channels, $\big(h[b, t, c],\ h[b, t, c']\big)$ for $c' \neq c$. Per anchor, $C - 1$ candidates.
- **Cross-batch** ($\mathcal{N}_{\mathrm{cb}}$): the future latent of one series paired with the forecast of a different series at the same prediction position, $\big(h[b, t+1, c],\ f[b', t, c]\big)$ for $b' \neq b$. Per anchor, $B - 1$ candidates.

Many other pairings are conceivable. The cross-temporal axis can also be applied to $\big(h[b, t, c],\ f[b, t, c]\big)$ or $\big(f[b, t, c],\ f[b, t+1, c]\big)$. The cross-batch axis can also be applied to $\big(h[b, t, c],\ h[b', t, c]\big)$, $\big(f[b, t, c],\ f[b', t, c]\big)$, or $\big(h[b, t, c],\ f[b', t, c]\big)$. Any of these can be combined with a channel difference. In general, adding more terms to the denominator gives a faster loss decrease per backward pass at the cost of slower loss computation, and the trade-off is not always favourable; we specify the subset used in our experiments in Section 4.

Cross-channel candidates are computed by a $C \times C$ inner-product grid per batch element (diagonal masked), cross-batch candidates by a $B \times B$ grid over the whole batch (diagonal masked), and cross-temporal candidates by a single elementwise dot product over the encoded tensor. At every position (t, c), the candidates are aggregated into three sums of size B ($\mathcal{N}_{\mathrm{ct}}$), $B(C-1)$ ($\mathcal{N}_{\mathrm{cc}}$), and $B(B-1)$ ($\mathcal{N}_{\mathrm{cb}}$), and the same denominator is used for every anchor at that position. The cross-batch set is therefore quadratic in B and dominates the negatives.

For unit-norm $u, v \in \mathbb{R}^H$, write the temperature-scaled exponential of the inner product as

$$\sigma_\tau(u, v) \;=\; \exp\big(\langle u, v\rangle / \tau\big).$$

The positive score at anchor (b, t, c) is

$$\mathcal{P}_+(b,t,c) \;=\; \sigma_\tau\big(f[b,t,c],\, h[b,t+1,c]\big),$$

and the three negative sums at position (t, c) are

$$\begin{aligned}
\mathcal{N}_{\mathrm{ct}}(t,c) \;&=\; \sum_{b\in B} \sigma_\tau\big(h[b,t,c],\; h[b,t+1,c]\big),\\
\mathcal{N}_{\mathrm{cc}}(t,c) \;&=\; \sum_{b\in B}\sum_{c'\neq c} \sigma_\tau\big(h[b,t,c],\; h[b,t,c']\big),\\
\mathcal{N}_{\mathrm{cb}}(t,c) \;&=\; \sum_{b\neq b'} \sigma_\tau\big(h[b,t+1,c],\; f[b',t,c]\big).
\end{aligned}$$

The contribution of the anchor (b, t, c) to the loss is

$$\mathcal{L}_{b,t,c} \;=\; -\log \frac{\mathcal{P}_+(b,t,c)}{\mathcal{N}_{\mathrm{ct}}(t,c) \;+\; \mathcal{N}_{\mathrm{cc}}(t,c) \;+\; \mathcal{N}_{\mathrm{cb}}(t,c)}. \tag{1}$$

The denominator depends on (t, c) but not on b: every anchor at the same position shares the same denominator. The mini-batch loss is the average over all anchors,

$$\mathcal{L} \;=\; \frac{1}{N}\sum_{b\in B}\sum_{t}\sum_{c\in C} \mathcal{L}_{b,t,c}, \qquad N = B\cdot(T_p-1)\cdot C. \tag{2}$$

In the contrastive learning literature τ is usually chosen between 0.05 and 0.5; the value used in our experiments is reported in Section 4.

4 Experiments

We want to verify whether the backbone of Section 3, trained with the contrastive loss of Section 3.4, learns information about the process that generated the data and, in particular, whether the underlying parameters can be recovered from the latents. We do this with two controlled synthetic experiments where the generating parameters are known.[1]

4.1 ARMA

Training data. For each training sample we draw orders p, q uniformly from $\{1, \dots, P\}$, then sample the p AR coefficients and the q MA coefficients independently from one of two distributions chosen by a fair coin flip per sample: $\mathrm{Uniform}(-1, 1)$ or standard normal $\mathcal{N}(0, 1)$. We then rescale each coefficient set so that its ℓ_1 norm does not exceed 0.95: if $\sum_i |\phi_i| > 0.95$, we multiply every ϕ_i by $0.95/\sum_i |\phi_i|$, and similarly for θ. The bound $\sum_i |\phi_i| \leq 0.95$ is a sufficient stationarity condition for the AR polynomial $1 - \sum_i \phi_i L^i$ (and an invertibility condition for the MA polynomial $1 + \sum_i \theta_i L^i$). The coefficient vectors are then zero-padded up to length P, so that every sample has a fixed total of $2P$ ground-truth coefficients regardless of the realised orders. We evaluate two configurations: $P = 2$ (which we call *2x2*, 4 coefficients per sample) and $P = 4$ (*2x4*, 8 coefficients per sample); 2x2 serves as a smaller comparison point.

A series is generated by simulating $T = 4096$ steps from the sampled ARMA process driven by i.i.d. standard normal noise. We use the simplification $F = 1$ and $C = 1$ (each sample is a single univariate series). The series is then patched into $T_p = T/W = 128$ patches of width $W = 32$ following Section 3.1.

Training samples are drawn fresh at every step: every batch is a new draw of orders, coefficients and noise, and there is no fixed dataset of finite size.

[1] Code, training scripts and experiment notebooks are available at `https://github.com/jeremycochoy/contrastive-forecasting`.

Loss. Section 3.4 defines three negative families. In the present setting two of them simplify away.

- The cross-channel term $\mathcal{N}_{\mathrm{cc}}$ vanishes because $C = 1$: there is no other channel to draw negatives from.
- The cross-temporal term $\mathcal{N}_{\mathrm{ct}}$ is dropped by design. We want to learn the underlying parameters of the generating ARMA process, not the specific realised samples; successive samples in the same series are produced by the same parameters and the past directly influences the future, so a cross-temporal pair is two views of the same process, and pushing them apart in latent space would conflict with our goal.

Only the cross-batch term $\mathcal{N}_{\mathrm{cb}}$ remains. Dropping the channel index, the contribution of an anchor (b, t) to the loss is

$$\mathcal{L}_{b,t} \;=\; -\log \frac{\mathcal{P}_{+}(b,t)}{\mathcal{N}_{\mathrm{cb}}(t)}, \tag{3}$$

with

$$\mathcal{P}_{+}(b,t) \;=\; \sigma_{\tau}\big(f[b,t],\, h[b,t+1]\big), \qquad \mathcal{N}_{\mathrm{cb}}(t) \;=\; \sum_{b \neq b'} \sigma_{\tau}\big(h[b,t+1],\, f[b',t]\big).$$

We use $\tau = 0.07$.

Backbone. We use the backbone of Section 3 with $L = 12$ transformer layers and hidden width $H = 1024$. The encoder is the bidirectional GRU described in Section 3.2. Training runs for 2M steps with AdamW, learning rate 7×10^{-5}, batch size 8. The architecture-search history that motivates these choices is reported in Appendix D.

We monitor backbone training with the *contrastive gap*, defined as

$$\mathrm{gap} \;=\; \mathbb{E}_{b,t}\big[\langle f[b,t],\, h[b,t+1]\rangle \;-\; \langle f[b,t],\, h[b,t]\rangle\big],$$

the difference between the mean forecast-future and forecast-past cosine similarities on a held-out batch. A higher gap means the forecast latent is closer to the future patch at $t+1$ than to the past patch at t. The gap is a diagnostic, not optimised directly; it tracks well with downstream recovery quality and is the metric we use to compare backbone variants in Appendix D.

Recovery head. To probe whether the backbone latents encode the ARMA parameters, we freeze the backbone after training and fit a small *recovery head* on top of it. The head receives the per-patch latent $h[b,t] \in \mathbb{R}^H$ produced by the frozen backbone and returns a per-patch coefficient prediction $\hat{\theta}[b,t] \in \mathbb{R}^{2P}$.

Internally the head has four parts: an input projection (linear, GELU, layer norm) mapping $\mathbb{R}^H \to \mathbb{R}^{128}$, a bidirectional GRU with hidden size 128 and 2 layers, a two-layer GELU MLP applied to the GRU output, and two parallel linear heads (one for AR, one for MA), each followed by $\tanh$ to constrain outputs to $(-1, 1)$ in line with the sampling range. The full head has roughly 676k parameters. The single coefficient prediction for a series is the mean of the per-patch predictions:

$$\hat{\theta}[b] \;=\; \frac{1}{T_p} \sum_{t=1}^{T_p} \hat{\theta}[b,t].$$

The head is trained with mean squared error against the ground-truth coefficients of each sample. The architecture-search history that led to this head is reported in Appendix C.

We evaluate the head with the *improvement ratio*

$$r \;=\; \frac{\mathrm{MSE}_0}{\mathrm{MSE}_{\hat{\theta}}}, \tag{4}$$

where MSE_0 is the MSE of the all-zero baseline that predicts $\hat{\theta} = 0$ for every coefficient. This baseline is non-trivial: since order padding leaves a fraction of the ground-truth coefficients at exactly zero, the all-zero predictor is already perfect on those positions and so achieves a fairly low MSE. The head must do better still in order to reach $r > 1$.

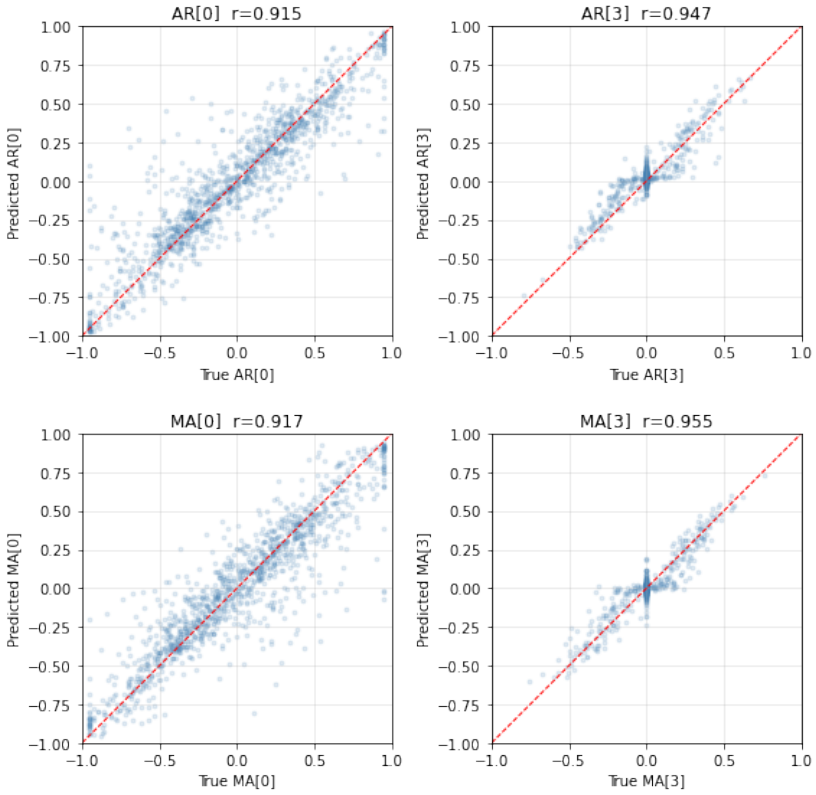

Figure 1: Recovery quality on the 2x4 (ARMA(4, 4)) configuration: predicted vs ground-truth coefficient for the four extreme positions AR_0, AR_3, MA_0, MA_3. Each subplot is a scatter of 300 test samples, with the identity reference line and the Pearson correlation r between predicted and true coefficients.

4.2 Random-walk correlations

Data. For each training sample we draw a 4×4 correlation matrix Σ by sampling the six off-diagonal entries i.i.d. from $\mathrm{Uniform}(0, 1)$ and rejecting non-PSD draws. A series of $C = 4$ channels of length $T = 4096$ is generated by cumulating i.i.d. increments from $\mathcal{N}(0, \Sigma)$; each channel is z-scored within the sample.

Loss. To enable cross-channel mixing inside the backbone, all C per-channel encodings at each timestep are projected jointly to $\mathbb{R}^H$ before the transformer (effective $C = 1$). Consequently $\mathcal{N}_{\mathrm{cc}}$ vanishes and $\mathcal{N}_{\mathrm{ct}}$ is dropped for the same reason as in the ARMA case: only $\mathcal{N}_{\mathrm{cb}}$ is active.

Backbone and recovery head. The backbone uses $L = 12$, $H = 1024$, batch size 16, AdamW at 7×10^{-5}, 195 000 steps. The recovery head is the same architecture as Section 4.1 with output dimension 6 and $\mathrm{clamp}(0, 1)$ in place of $\tanh$, trained with MSE against the six entries of Σ.

5 Results

5.1 ARMA

The recovery head reaches $r = 8.34$ on the smaller 2x2 configuration and $r = 6.96$ on the larger 2x4 configuration. Both numbers are well above 1, the threshold above which the head's predictions improve on the all-zero baseline.

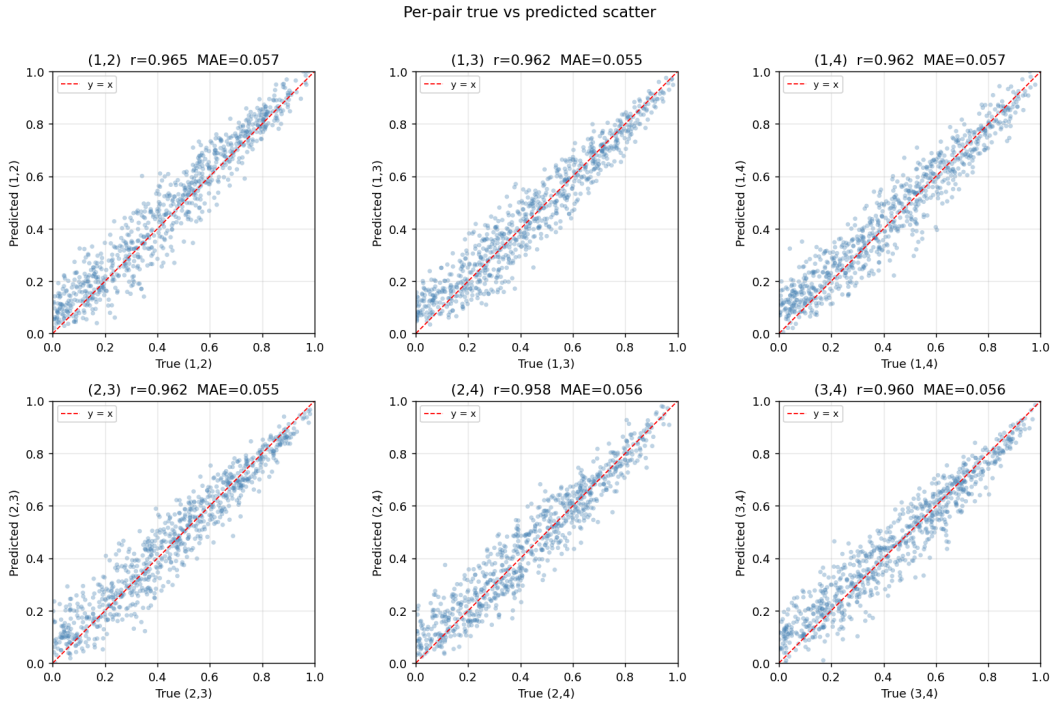

Figure 2: Per-pair recovery for the random-walk correlation experiment: predicted vs. true correlation for each of the six off-diagonal entries of Σ. Each subplot is a scatter of 800 test samples with the identity reference line and Pearson r. All six pairs lie in $r \in [0.958, 0.964]$.

Figure 1 shows the predicted vs ground-truth coefficient on the 2x4 setup for the four extreme positions: AR_0, AR_3, MA_0, MA_3. The Pearson correlation r between predicted and true coefficients sits between 0.91 and 0.96 on these four positions.

The full 8-coefficient scatter for 2x4, the equivalent for 2x2, the per-sample error distributions, and the per-coefficient table are reported in Appendix A.

5.2 Random-walk correlations

The recovery head achieves a mean Pearson $r = 0.962$ across all six pairwise correlations on an 800-sample test set.

Figure 2 shows the per-pair scatter; all six pairs lie in $r \in [0.958, 0.964]$. Full results are in Appendix B.

6 Conclusion

Inspired by the JEPA principle of learning an abstract representation rather than the raw values of the input (the pixels of an image, the samples of a time series), we have demonstrated the feasibility of training a forecasting model directly in latent space, in an unsupervised manner (Section 4). The latents produced by the backbone carry enough information about the generating stochastic process for a small head to recover its parameters (Section 5), at least in the ARMA case.

The synthetic distribution is the main limit of these results. ARMA processes capture only one slice of what real time series can look like, so the recovery numbers do not say anything about how the same backbone behaves on real-world data, such as electricity consumption, oil prices, monthly sales, or road traffic. The natural next step is to evaluate it on a general time-series forecasting benchmark.

Acknowledgements

Claude Code (Anthropic Claude Opus 4.7) assisted in drafting this manuscript and in running some of the experiments, under the author's direction. The author takes full responsibility for the contents of this paper.

References

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with TiDE: Time-series dense encoder. *Transactions on Machine Learning Research*, 2023.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *International Conference on Machine Learning*, 2024.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

Sofiane Ennadir et al. Joint embeddings go temporal. *arXiv preprint arXiv:2509.25449*, 2025.

Jean-Yves Franceschi, Aymeric Dieuleveut, and Martin Jaggi. Unsupervised scalable representation learning for multivariate time series. In *Advances in Neural Information Processing Systems*, 2019.

Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. MOMENT: A family of open time-series foundation models. In *International Conference on Machine Learning*, 2024.

Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Sundial: A family of highly capable time series foundation models. *arXiv preprint arXiv:2502.00816*, 2025.

Yong Liu et al. Timer-S1: A scalable time series foundation model with serial-token prediction. *arXiv preprint arXiv:2603.04791*, 2026.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.

Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020.

David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3): 1181–1191, 2020.

Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-MoE: Billion-scale time series foundation models with mixture of experts. *arXiv preprint arXiv:2409.16040*, 2024.

Sana Tonekaboni, Danny Eytan, and Anna Goldenberg. Unsupervised representation learning for time series with temporal neighborhood coding. In *International Conference on Learning Representations*, 2021.

Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Stijn Verdenius, Andrea Zerio, and Roy L. M. Wang. LaT-PFN: A joint embedding predictive architecture for in-context time-series forecasting. *arXiv preprint arXiv:2405.10093*, 2024.

Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *International Conference on Machine Learning*, 2024.

Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. TS2Vec: Towards universal representation of time series. In *AAAI Conference on Artificial Intelligence*, 2022.

A Full result graphs

Figure 3 shows the full 8-coefficient scatter for the 2x4 setup: each subplot is a scatter of 300 test samples with the identity reference line and the Pearson correlation between predicted and true coefficients. The Pearson r values range from roughly 0.92 to 0.96 across the eight coefficients.

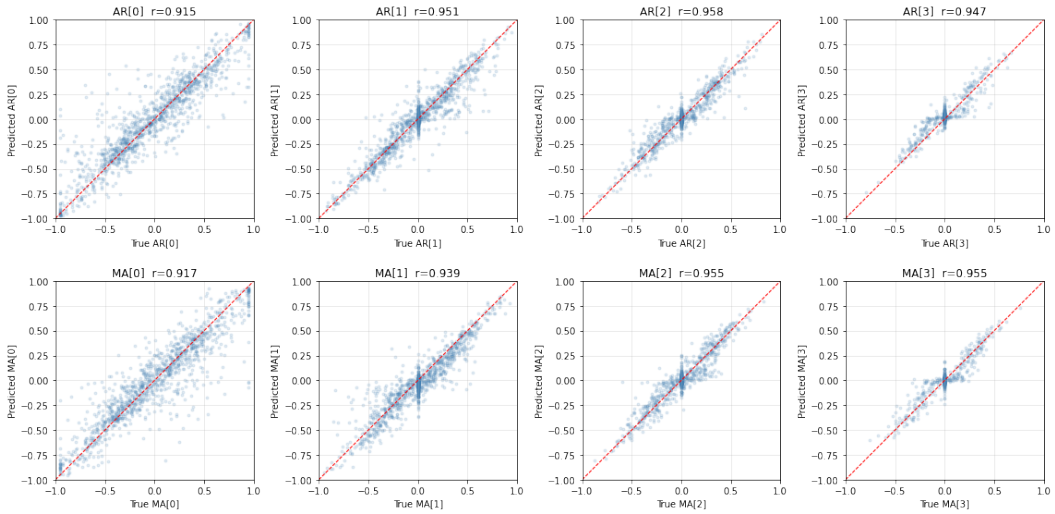

Figure 3: Full 8-coefficient scatter for the 2x4 (ARMA(4, 4)) configuration: AR_0 to AR_3 on the top row, MA_0 to MA_3 on the bottom row.

Figure 4 shows the equivalent 4-coefficient scatter for the 2x2 setup. The Pearson r values range from 0.94 to 0.97, slightly higher than for the 2x4 case, consistent with the higher overall recovery ratio of 8.34.

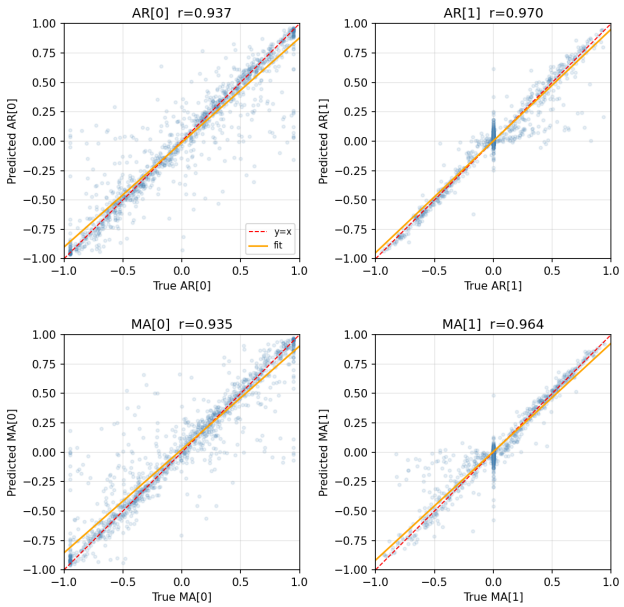

Figure 4: Full 4-coefficient scatter for the 2x2 (ARMA(2, 2)) configuration: AR_0 and AR_1 on the top row, MA_0 and MA_1 on the bottom row.

Figure 5 shows the per-sample MSE distribution for the 2x4 head on 200 test samples.

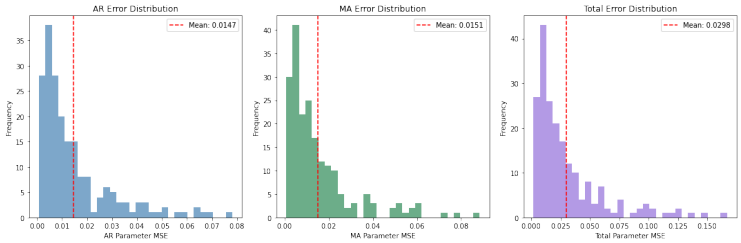

Figure 5: Per-sample MSE distributions for the recovery head on the 2x4 configuration.

B Random-walk correlation: full results

Figure 6 shows ground-truth and predicted 4×4 correlation matrices for a random selection of test samples.

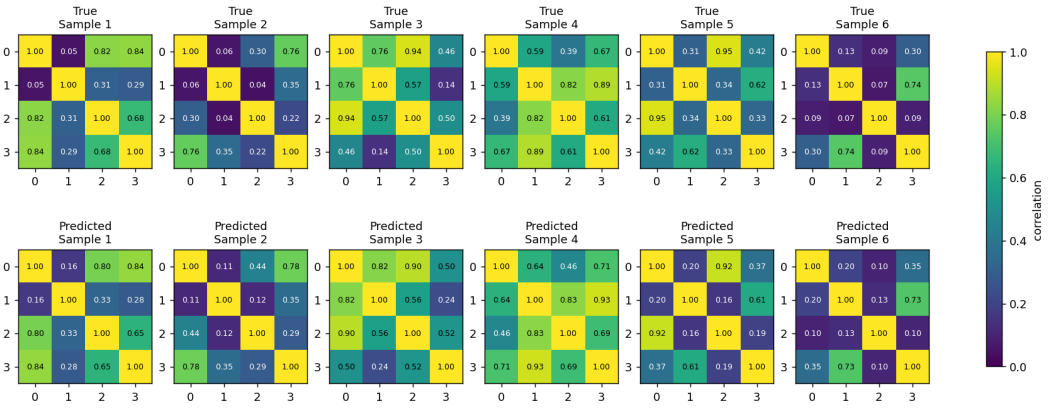

Figure 6: Predicted (left half) vs. true (right half) correlation matrices for six test samples drawn from the 800-sample test set.

Figure 7 shows the per-sample MSE distribution on the 800-sample test set.

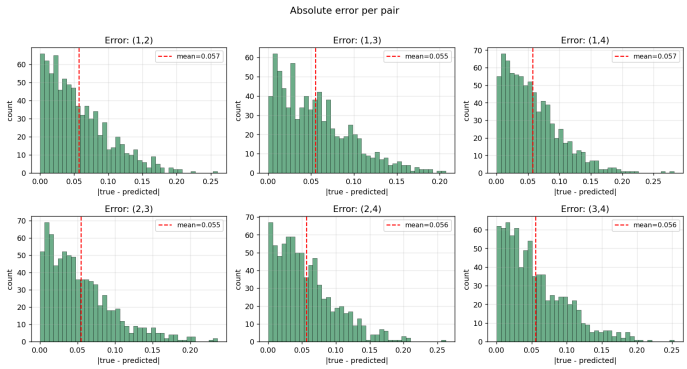

Figure 7: Per-sample MSE distribution for the correlation recovery head on the 800-sample test set.

C Recovery head architecture search

We searched the recovery head architecture across approximately 47 configurations spanning four axes: head type (MLP, residual MLP, single GRU, deep GRU, GRU with pooling, attention pooling), hidden width in $\{128, 256, 512\}$, depth in $\{1, 2, 3, 4\}$ layers, and loss function (MSE, Huber, L1, weighted MSE).

The best head is the simple bidirectional GRU with hidden size 128 and 2 layers, trained with MSE. It outperforms a 4.7M-parameter "DeepGRU" baseline by a clear margin while using roughly $7\times$ fewer parameters, and outperforms wider variants at $h \in \{256, 512\}$. Adding more than 2 layers degrades recovery. Among loss functions, MSE is indistinguishable from Huber on the converged metric and slightly better than L1 and weighted MSE.

D Backbone architecture search

The backbone we use was selected through two phases of search.

Encoder. The first phase compared five encoder variants holding the rest of the backbone fixed: a TiDE-style residual MLP, a wider MLP variant, a TimeFM-style residual SiLU block, a 1-D convolution, and a bidirectional GRU. The bidirectional GRU won by a wide margin, with roughly a 62% relative increase in the contrastive gap over the MLP baseline at 50k steps. Each patch is read as

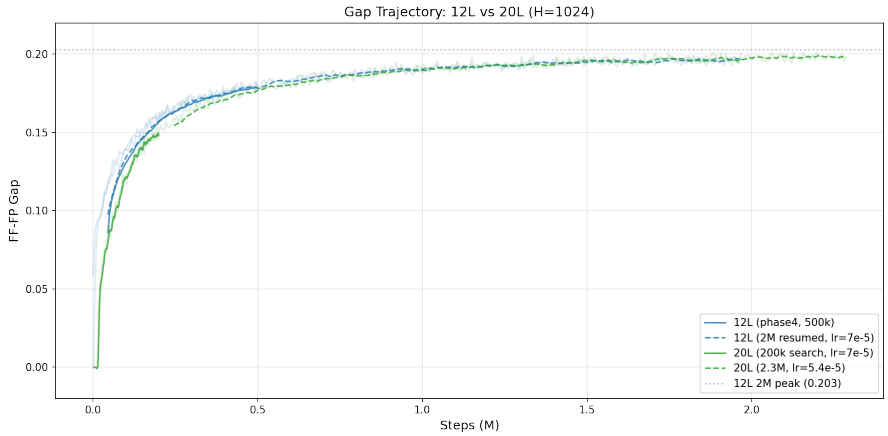

Figure 8: Contrastive gap (forecast–future minus forecast–past) over training steps for the 12L (extended to 2M steps) and 20L (extended to 2.3M steps) backbones at $H = 1024$.

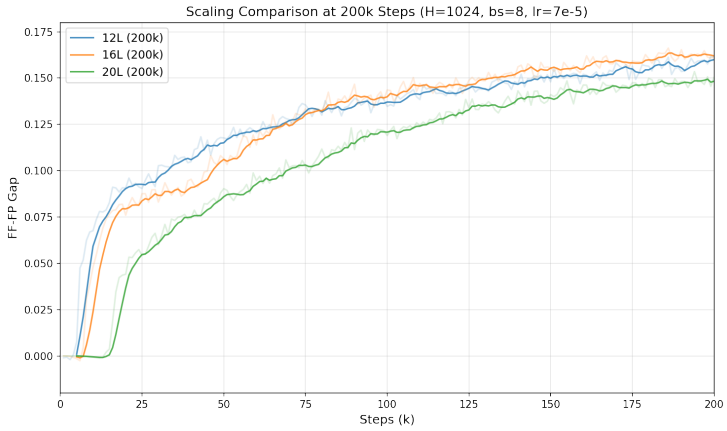

Figure 9: Contrastive gap at 200k training steps for $L \in \{12, 16, 20\}$ at $H = 1024$.

a sequence of W scalar timesteps, which gives the GRU access to the local temporal structure that an MLP collapses into a single linear projection.

Forecaster. The second phase varied the transformer block. The depthwise causal convolution of Section 3.3 is essential: removing it drops the converged gap by roughly 20%. A feed-forward multiplier of 4 outperforms 2 by a few percent. GELU outperforms SiLU at the same parameter count. Doubling the number of attention heads from 8 to 16 gave no measurable gain.

Depth. We compared $L \in \{12, 16, 20\}$ at $H = 1024$. At 200k steps the 16L variant led; the 20L variant trained more slowly and lagged behind early on. After the full 2M steps, the 20L matched the 12L on the contrastive gap but came out slightly worse on recovery (6.77 vs 6.96). We attribute this lag to imperfect learning-rate scaling for the deeper model rather than to a depth-vs-shallow finding, and use $L = 12$ throughout our runs as the compute-efficient choice that does not lose recovery quality.

Figure 8 shows the training gap trajectories for the 12L and 20L variants over the full training run. Figure 9 shows the 200k snapshot comparison across $L \in \{12, 16, 20\}$.